

Bilan critique et revue de code

Bilan critique

Analyse du livrable reçu

La description de l'application était vraiment très claire et complète. Nous avons pu comprendre le fonctionnement de l'application sans souci. Le seul doute était de savoir pourquoi dans le code l'élève se connectait avec son mail alors que l'intervenant se connectait avec un login. Cette asymétrie se retrouve directement dans les signatures des services (`eLeveLogin(mail, mdp)` et `intervenantLogin(login, mdp)`), donc ce n'était pas une erreur de lecture de notre part mais bien un choix de modèle qui aurait mérité d'être explicité dans la description.

Le reste du dossier complétait très bien la description. L'IHM a surtout été d'une grande utilité, parce qu'elle indiquait sur le dessin (en plus du tableau ICARS) l'enchaînement des fenêtres. Mais les autres parties étaient aussi bien faites avec les tableaux ICARS qui indiquaient bien les services à appeler. La description des services était aussi très claire et fidèle au code, évitant ainsi des recherches dans le code pour connaître le fonctionnement ou la signature d'un service.

Le seul défaut que nous pourrions citer pour le livrable est la petite inconsistance pour la durée d'une intervention qui doit être indiquée par l'intervenant à la fin de la séance mais qui n'était pas marquée dans l'IHM.

Analyse rétrospective

Nous aurions pu plus étoffer et reformuler la description de l'application. C'est possible que nous étions induits en erreur par notre niveau de compréhension. Certaines parties nous paraissaient peut-être plus évidentes à cause du nombre d'heures que nous avons passé dessus.

De plus, même si nous considérons que nos IHM étaient claires, les images prenaient peut-être trop de place pour avoir une image globale de l'application. L'idée d'avoir une page avec l'enchaînement de fenêtres est vraiment pas mal. Cela nous a aussi conduit à avoir trop de pages.

Une des choses que nous avons bien fait, est le fait de mettre en avant (en gras ou avec des couleurs) les informations les plus importantes.

Revue de code

Le code fonctionnait et était facile à comprendre lorsque nous avons besoin de le lire.

La majorité des services dont nous avons besoin étaient implémentés. L'inscription d'un élève et la création d'une demande étaient parfaitement implémentées. Ça aurait été plus simple si la création d'une demande renvoyait la demande créée, et si le lien de la vidéoconférence était généré pour qu'on ne mette pas de lien bidon. Mais, on s'en est sorti sans.

Cependant nous avons dû en rajouter certains, dont les `getEntityById()` dont nous avons besoin pour pouvoir simplement retenir dans la session l'id de l'élève ou de l'intervenant connecté. De plus, nous en

avons aussi besoin pour pouvoir donner les id de demande par exemple en paramètre d'URL pour la prochaine page.

De plus, certains services nécessitaient des corrections pour qu'ils fonctionnent correctement. Par exemple, pour la connexion, si la personne qui essayait de se connecter n'existait pas ou s'était trompée de mot de passe (le DAO ne trouve pas d'entité correspondante), le service mettait une erreur parce qu'il essayait d'accéder au mot de passe d'un objet nul. Nous avons alors rajouté une vérification pour voir si l'objet n'est pas `null`. Le fait que ça renvoie un objet `null` était implémenté dans le `EleveDao` mais pas dans le `IntervenantDao`. Pour nous faciliter les tests, nous l'avons alors ajouté.

Nous avons aussi rencontré un bug dans la requête `findIntervenant` du `IntervenantDao` : les comparateurs sur le niveau étaient inversés (`niveau_min >= :niveau AND niveau_max <= :niveau` au lieu de l'inverse). En l'état, la requête ne pouvait quasiment jamais renvoyer un intervenant valide, ce qui faisait échouer la création de demande sans raison apparente.

Et enfin, nous avons des problèmes pour avoir l'intervention en cours. En effet, l'attribut qui devrait avoir l'historique des interventions ne donnait pas de valeur cohérente. Heureusement, un service qui permettait de récupérer l'historique des interventions (de l'élève et de l'intervenant) existait. Nous avons alors modifié le DAO pour qu'il ne nous donne pas que les interventions terminées mais toutes. Cela nous a permis de récupérer l'intervention en cours avec une boucle qui vérifiait si l'intervention était terminée ou pas.

De manière beaucoup moins grave, l'attribut `nbIntervention` de l'intervenant ne se mettait pas à jour lors de la clôture d'une intervention. En regardant de plus près, `closeDemande` modifie l'intervenant (`setDisponible`, `setNb_intervention`) avant l'ouverture de la transaction, et appelle même `intDao.update(it)` deux fois. La modification se fait donc sur un objet en partie détaché du contexte de persistance, ce qui explique probablement que la mise à jour ne soit pas prise en compte. Le même type d'incohérence (contexte ouvert/fermé puis ré-ouvert avec un objet conservé entre les deux) se retrouve dans `creationDemande`.

Enfin, un point de rétrospective plus général : la plupart des services renvoient juste un booléen sans préciser la cause de l'échec (établissement introuvable, aucun intervenant disponible, mot de passe incorrect...). Côté IHM, ça nous a obligés à afficher des messages d'erreur génériques alors qu'un code de retour plus riche aurait permis de mieux guider l'utilisateur.